LAB No 14.

Objective: Open Ended Lab

PROBLEM STATEMENT:
Design and Implement a custom Database Storage Engine for a Relational DBMS or a Key Value Store.

PROBLEM DESCRIPTION:
In this laboratory assignment, you are tasked with designing and implementing a custom Database Storage Engine for a Relational DBMS or a Key Value Store. The goal is to demonstrate your understanding of database architecture design principles, data storage strategy, query processing and optimization, and practical implementation of the operations in a database storage engine.

INSTRUCTIONS:

1. Select a real world database storage engine that can be implemented as a custom storage engine. Examples include a relational DBMS or a key value store. Ensure that the chosen application is sufficiently complex to showcase your skills in database design and implementation.
2. Design the database architecture for the selected storage engine. Start by identifying the various components of the architecture including query parser (interprets user commands), schema manager (maintains structure and metadata), storage engine (handles read and write), query processor (executes database operations), index manager [optional] (supports optimized querying). Provide an architecture diagram illustrating the interaction between these modules.
3. Create a data storage strategy as per the selected storage engine and the architectural components. For relational storage engines, store tabular data in .csv files, maintain the first row as attribute headers, ensure structured storage and parsing. For NoSQL based systems, store data in .json files, each document should follow a structured key value format. Describe how file reading and writing operations are managed.
4. Implement a database handle which is returned when a connection is opened. If db_open is successful, the specified files are created in the system. This handle is passed to other database functions. A flag parameter is used to specify the format of the file when it is opened (read only, read write, create file if it doesn't exist). When the querying is complete, we call db_close to close the index file and the data file and release memory that it allocates for internal buffers.
5. Populate the database with representative data to simulate real world scenarios. Ensure that the data reflects various aspects of the selected application, such as different user profiles, transactions, or entities.
6. Develop SQL queries that showcase the functionality of the DBMS. Include queries for data insertion, data retrieval, data updates, and complex operations that highlight the capabilities of the DBMS and demonstrate your proficiency in SQL.
7. Optimize the query performance by analyzing query execution plans, creating appropriate indexes, and utilizing database optimization techniques such as query rewriting, implementing B+Tree indexing, or hashing techniques. Implement data validation and integrity constraints with data type validations, Unique primary key enforcement, and required field validation.
8. Test and validate the database system by executing the implemented queries and performing various operations on the database. Verify that the system functions correctly, handles reads and writes, and maintains data consistency in files.
9. Document your database architecture design, implementation decisions, and any challenges faced during the process. Provide a detailed explanation of the architecture, storage strategy, constraints, and optimizations performed.
10. Present your database storage engine system, highlighting its features, performance, and usability.